

Fundamental Machine Learning using Python for Public Health

Python Foundations, Feature Engineering & Your First Machine Learning Models

Teerapong Panboonyuen, Ph.D. (Kao)

College of Computing, Khon Kaen University

What We'll Build Today

1

A • Python Refresher

Variables, operators, conditionals, loops, containers, functions, classes — the exact building blocks used in every data-science script.

2

B • Pandas & EDA

Load the KGU Stroke Dataset, run the 3 golden checks, and explore distributions, imbalance, and relationships with the target.

3

C • Feature Engineering

Imputation, clinical binning, encoding, composite risk scores — think like an epidemiologist and a data scientist at once.

4

D • ML Pipeline

Stratified split → Decision Tree → Random Forest → feature importance.

5

E • Evaluation

Accuracy, Precision, Recall, F1, ROC-AUC, ROC curves, and confusion matrices — read like a reviewer.

6

F • Recap

What is supervised learning, really? A clean summary before we go deeper tomorrow.

Learning Objectives

- ✓ Confidently use core Python building blocks for data-science tasks.
- ✓ Load, clean, and explore a real clinical dataset with pandas.
- ✓ Engineer new features from raw health data the way a data scientist would.
- ✓ Build a complete supervised learning pipeline with scikit-learn.
- ✓ Train and compare a Decision Tree and a Random Forest to predict stroke risk.
- ✓ Read and interpret feature importance, ROC curves, and a confusion matrix like a researcher.

The KGU Stroke Dataset

1

Prediction Task

Binary classification — will a patient have a stroke (1) or not (0)? A rare, high-stakes clinical outcome.

2

Size

5,110 patient records × 12 columns: demographics, clinical history, lifestyle factors, and physiological measurements.

3

Source

kku-stroke-dataset.csv — teaching version hosted on GitHub, credit: Kaggle (Stroke Prediction Dataset).

4

Challenge

Missing BMI values, a highly imbalanced target, and mixed numeric/categorical features — realistic, messy clinical data.

Data Dictionary

Variable	Type	Description
id	Integer	Unique participant identifier
gender	Categorical	Male / Female / Other
age	Numeric	Age in years
hypertension	Binary	History of hypertension (0/1)
heart_disease	Binary	History of heart disease (0/1)
ever_married	Categorical	Marital status (Yes/No)
work_type	Categorical	Private, Self-employed, Govt_job, children, Never_worked
Residence_type	Categorical	Urban / Rural
avg_glucose_level	Numeric	Average blood glucose (mg/dL)
bmi	Numeric	Body Mass Index (kg/m ²) — has missing values
smoking_status	Categorical	never / formerly smoked / smokes / Unknown
stroke (target)	Binary	0 = No Stroke, 1 = Stroke



FOUNDATIONS

Python Refresher for Data Science

A speed-run through the exact Python building blocks you'll use, over and over, in every cell of this workshop.

Variables & Data Types

Type	Example	Used for...
int	age = 67	Counts, categorical codes
float	bmi = 36.6	Measurements (glucose, BMI)
str	gender = "Male"	Categorical / text data
bool	has_stroke = True	Binary flags, filters

Operators

```
glucose = 228.69

# Arithmetic
glucose / 10      # 22.869
glucose // 10     # 22.0  (floor division)
glucose % 10      # 8.69  (remainder)
glucose ** 2      # power

# Comparison + Logical
is_high = glucose > 200
is_senior = age >= 65
is_high and is_senior
```

Three families you'll use daily

- Arithmetic (+ - * / // % **) — computing derived numeric features.
- Comparison (== != > < >= <=) — the exact logic behind clinical thresholds.
- Logical (and / or / not) — combining multiple risk conditions into one flag.

Conditionals — Encoding Clinical Rules

```
def bmi_category(bmi_value):  
    """WHO adult BMI categories."""  
    if bmi_value < 18.5:  
        return "Underweight"  
    elif bmi_value < 25:  
        return "Normal"  
    elif bmi_value < 30:  
        return "Overweight"  
    else:  
        return "Obese"  
  
bmi_category(36.6)    # "Obese"
```

Why this matters

- if / elif / else lets us translate WHO / ADA clinical guidelines directly into code.
- Every threshold you'll bin later (BMI, glucose, age) follows exactly this pattern.
- This is the seed of Part C's Feature Engineering section.

Loops — Processing Many Records at Once

```
ages = [67, 61, 80, 49, 79, 12, 5]

# for-loop
seniors = []
for a in ages:
    if a >= 65:
        seniors.append(a)

# Pythonic list comprehension
seniors = [a for a in ages if a >= 65]

# while-loop
countdown = 3
while countdown > 0:
    countdown -= 1
```

The workhorse of data processing

- for-loops scan every row of a dataset — the same idea pandas uses internally.
- List comprehensions are the idiomatic, fast way to filter/transform data in Python.
- while-loops repeat until a condition changes — used in training loops on Day 2.

Containers: list, tuple, dict, set

Container	Ordered? / Mutable?	Typical ML use
list	Ordered · Mutable	Rows, feature lists, results
tuple	Ordered · Immutable	Fixed records, e.g. X_train, X_test = ...
dict	Insertion-ordered · Mutable	Mapping categories → codes, storing results
set	Unordered · Mutable	Unique categories, fast membership tests

Functions — Reusable, Testable Logic

```
def glucose_category(glucose_value):  
    """ADA-style average glucose categories (mg/dL)."""  
    if glucose_value < 140:  
        return "Normal"  
    elif glucose_value < 200:  
        return "Prediabetes"  
    else:  
        return "Diabetes_range"  
  
def risk_flag(age, hypertension, heart_disease,  
             age_threshold=65):  
    """A simple composite risk flag."""  
    return (int(age >= age_threshold)  
            + hypertension + heart_disease)
```

Inputs → logic → output

- Parameters are the inputs; return is the output — the fundamental unit of reuse.
- Default arguments (age_threshold=65) make functions flexible without extra calls.
- Docstrings document intent — good practice for reproducible research code.

Classes — Just Enough OOP for Data Science

```
class Patient:
    def __init__(self, patient_id, age, bmi):
        self.patient_id = patient_id
        self.age = age
        self.bmi = bmi

    def is_high_risk(self):
        return self.age >= 65 and self.bmi >= 30

p1 = Patient(9046, age=67, bmi=36.6)
p1.is_high_risk()  # True
```

This IS the scikit-learn pattern

- `__init__` sets attributes when an object is created — like `DecisionTreeClassifier(max_depth=5)`.
- Methods act on the object's own data — like `model.fit(X, y)` and `model.predict(X)`.
- You don't need to master OOP — just recognize this pattern everywhere in ML code.

Exercise Recap — Python Warm-up

- ✓ Q1 (Easy): `is_smoker(status)` → True if status is "smokes" or "formerly smoked".
- ✓ Q2 (Medium): filter ages ≥ 18 using a list comprehension → `adult_ages`.
- ✓ Q3 (Harder): `age_group(age)` → "Child" / "Adult" / "Senior", applied across a list with a loop.

Full worked solutions are provided in the companion Jupyter notebook.

B

REAL DATA

Loading & Exploring the Stroke Dataset with pandas

Time to move from toy examples to real, messy, clinical data.

The Three Golden Checks

1

`df.info()`

Column names, data types, and non-null counts
— your first sanity check on any new dataset.

2

`df.describe()`

Summary statistics (mean, std, min, max, quartiles) for every numeric column.

3

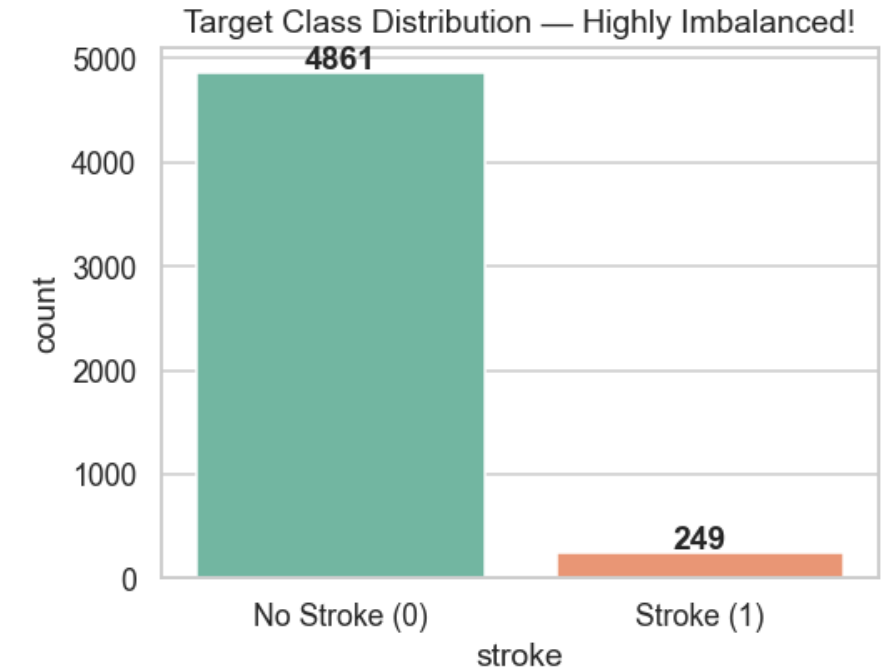
`df.isna().sum()`

Missing values per column — never silently drop rows in health data, you may lose sick patients!

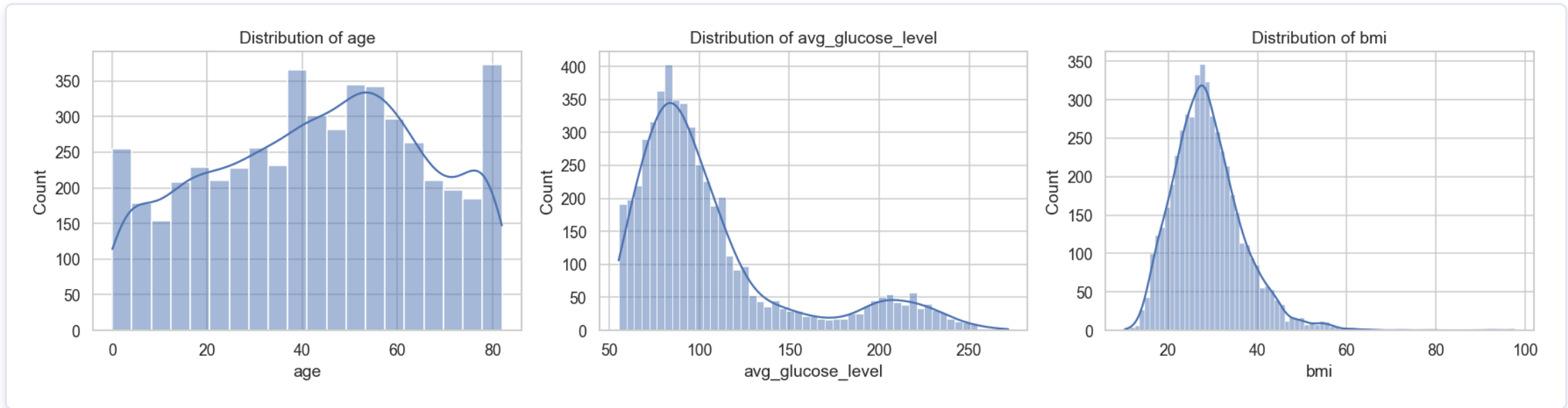
Is This Dataset Imbalanced?

Why this is the most important check

- Only ~5% of patients had a stroke — a classic rare-outcome clinical dataset.
- A lazy model that always predicts "no stroke" already scores ~95% accuracy — while being clinically useless.
- This is exactly why we will report Precision, Recall, F1 and ROC-AUC, not just accuracy.
- We will keep this same class ratio in both train and test sets via stratified splitting.

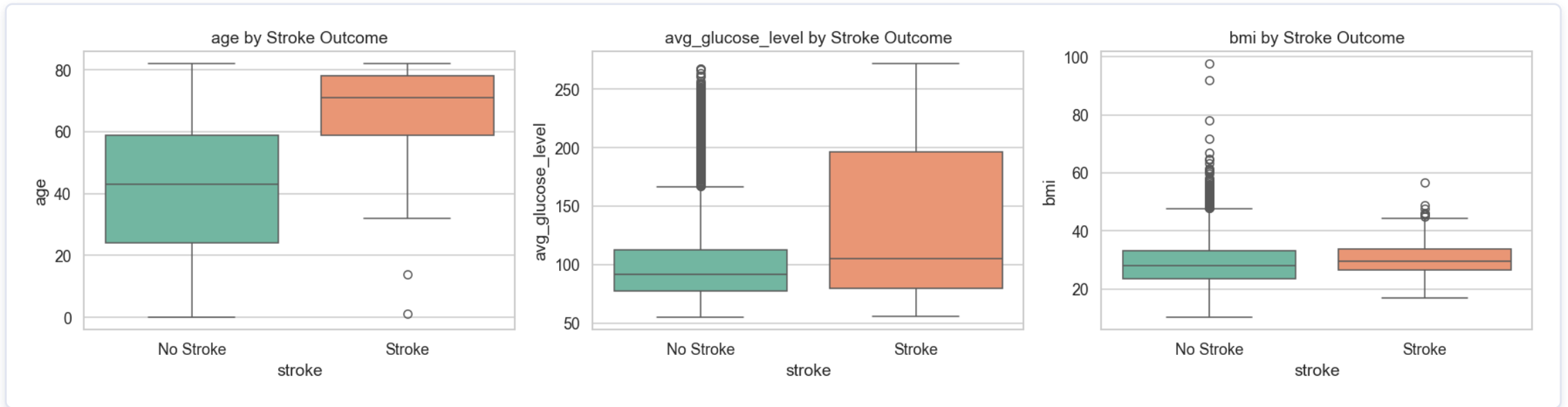


Distributions of Age, Glucose & BMI



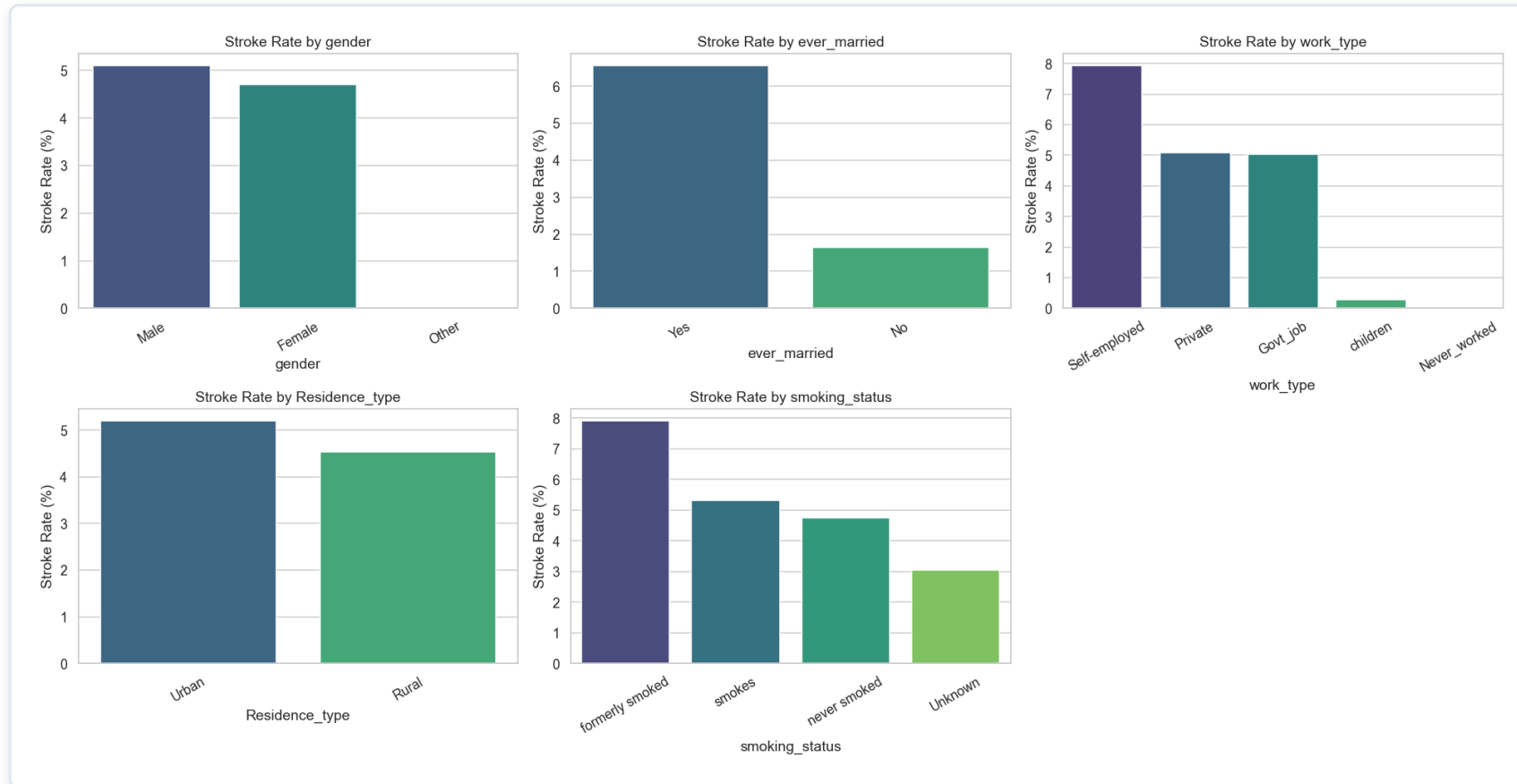
Age is broadly spread across adulthood; average glucose level and BMI are both right-skewed, with a long tail of high-risk values.

Numeric Features by Stroke Outcome



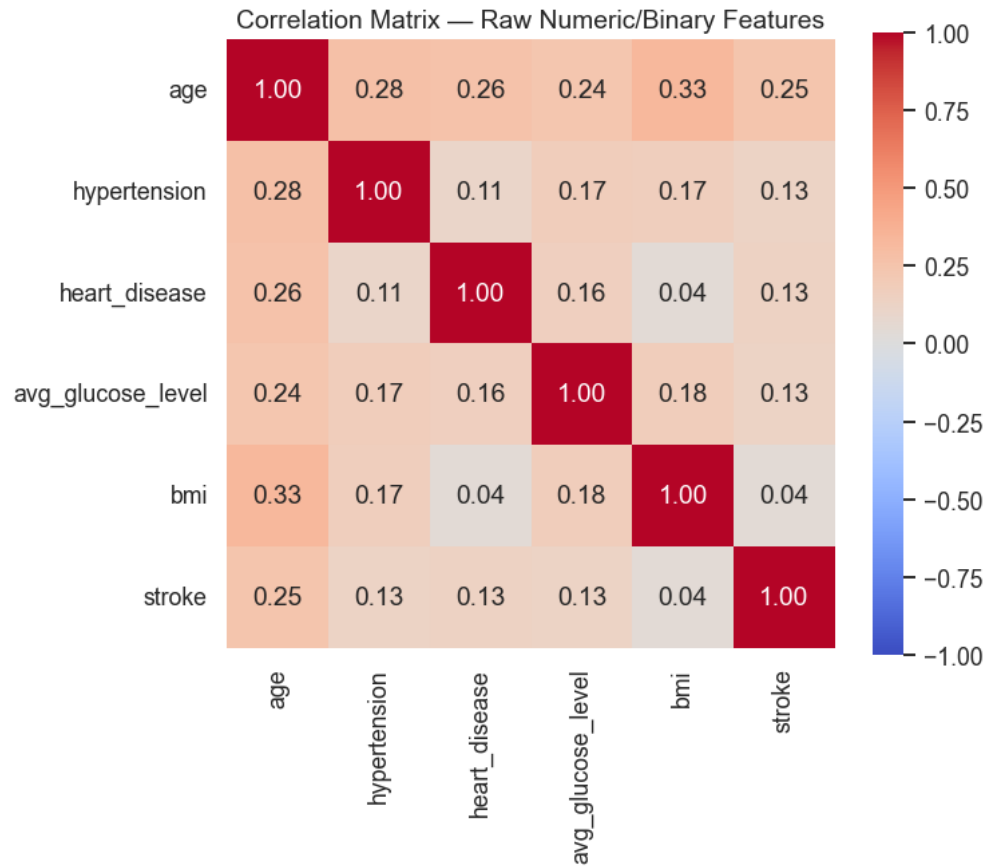
Patients with a stroke skew visibly older, with higher average glucose levels — early visual evidence age and glucose carry real predictive signal.

Stroke Rate by Categorical Variable



Stroke rate rises sharply with age-related categories, prior smoking, and being married — the kind of visual evidence you'd put in a paper's Results section.

Correlation Matrix



Reading the heatmap

- age shows the strongest raw correlation with stroke, consistent with decades of epidemiological literature.
- hypertension and heart_disease correlate with both age and stroke — expected comorbidity structure.
- No pair of features is extremely collinear, so we can safely keep them all for now.
- We'll revisit this exact matrix after feature engineering to check our new features.

A large, stylized orange letter 'C' is positioned on the left side of the slide, partially overlapping a dark blue circular shape in the top-left corner.

DOMAIN MEETS DATA SCIENCE

Feature Engineering — Thinking Like a Data Scientist

Raw columns are rarely the best model inputs. We transform them into features that expose the underlying clinical signal.

C.1 — Smarter Missing-Value Imputation

```
age_bin = pd.cut(data["age"],
                  bins=[0, 18, 40, 60, 80, 120])

data["bmi"] = data.groupby(
    [age_bin, "gender"]
)["bmi"].transform(
    lambda s: s.fillna(s.median())
)
data["bmi"] = data["bmi"].fillna(
    data["bmi"].median()    # safety net
)
```

Naive vs. expert imputation

- Naive: fill every missing BMI with one global median — loses real variation.
- Expert: fill with the median BMI of patients of similar age & gender — preserves more signal.
- Always add a safety-net fallback for any group with zero data available.

C.2 — Clinical Binning

Turning continuous numbers into clinically meaningful categories.

Feature	Rule	Categories
bmi_category	WHO adult BMI thresholds	Underweight / Normal / Overweight / Obese
glucose_category	ADA average glucose thresholds	Normal / Prediabetes / Diabetes_range
age_group	pd.cut into 5 clinical bands	0–18 / 19–40 / 41–60 / 61–80 / 81+

C.3 — Encoding Categorical Variables

1

Label Encoding

Map categories to integers (0, 1, 2...). Simple, but implies a false order — fine for tree models, risky for linear models.

2

One-Hot Encoding

One 0/1 dummy column per category. No false ordering — the standard, safe choice used throughout this workshop (`pd.get_dummies`).

3

Binary Mapping

`ever_married` (Yes/No) → a single 0/1 flag — the simplest and cleanest case.

4

Result

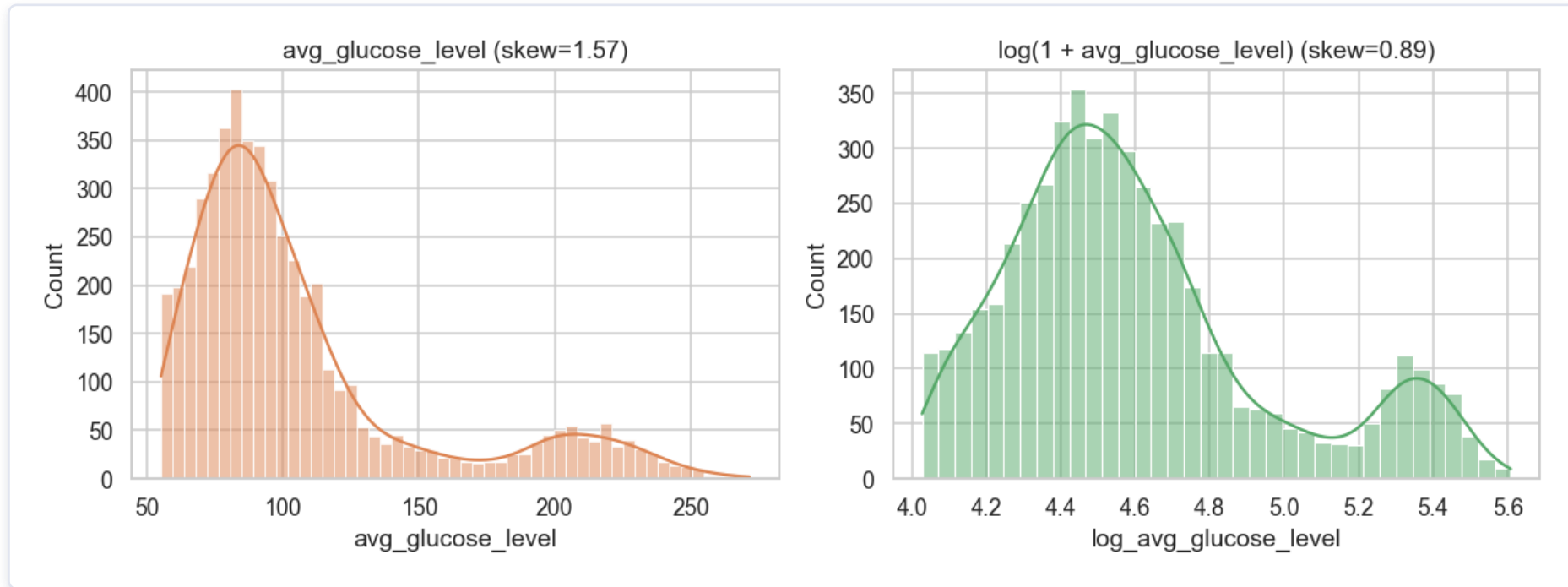
Our feature matrix grows from 12 raw columns to 37 engineered, model-ready columns.

C.4 — Composite & Interaction Features

Experts rarely rely on raw features alone — combining variables reveals patterns invisible to any single column.

New Feature	Formula	Clinical Intuition
comorbidity_count	hypertension + heart_disease	How many known risk conditions does this patient have?
age_x_hypertension	age × hypertension	Hypertension is riskier at older ages — captures interaction.
metabolic_risk_score	0.5·(glucose/max) + 0.5·(bmi/max)	A simple composite metabolic risk index.
is_senior	age ≥ 65	A highly interpretable binary marker used constantly in clinical papers.

C.5 — Skew Correction (Log Transform)

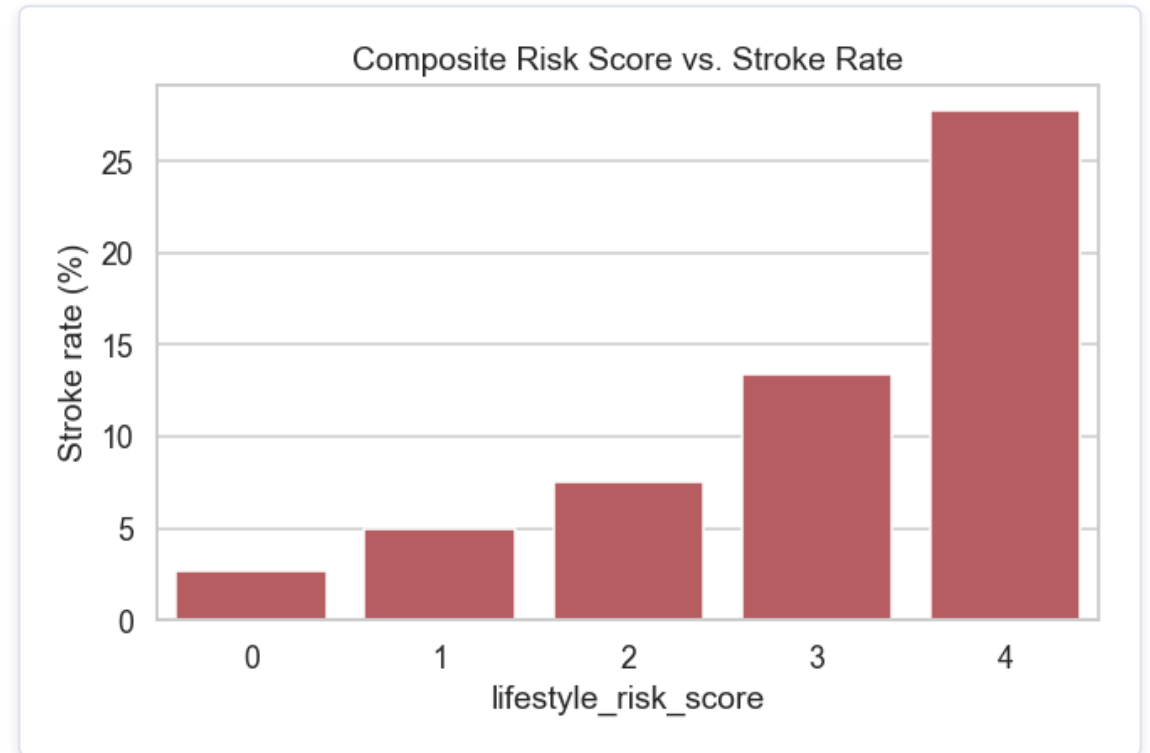


avg_glucose_level is right-skewed. A $\log(1 + x)$ transform compresses the long tail of very high glucose readings — this can help linear models on Day 2.

Exercise Recap — A Composite Lifestyle Risk Score

`lifestyle_risk_score` = `smoker_flag` + `high_bmi_flag` + `hypertension` + `heart_disease`

- Q1: `high_bmi_flag` = 1 if `bmi` ≥ 30 (WHO Obese threshold).
- Q2: `smoker_flag` = 1 if `smoking_status` is "smokes" or "formerly smoked".
- Q3: stroke rate rises steadily as the composite score climbs from 0 to 4 — validating the engineered feature.



D

MODELING

Your First Machine Learning Models

Stratified split → Decision Tree → Random Forest → feature importance → full evaluation.

D.1 — Stratified Train/Test Split

```
from sklearn.model_selection import (  
    train_test_split  
)  
  
X_train, X_test, y_train, y_test = (  
    train_test_split(  
        X, y,  
        test_size=0.25,  
        random_state=42,  
        stratify=y,    # keep class balance!  
    )  
)
```

Why stratify=y matters

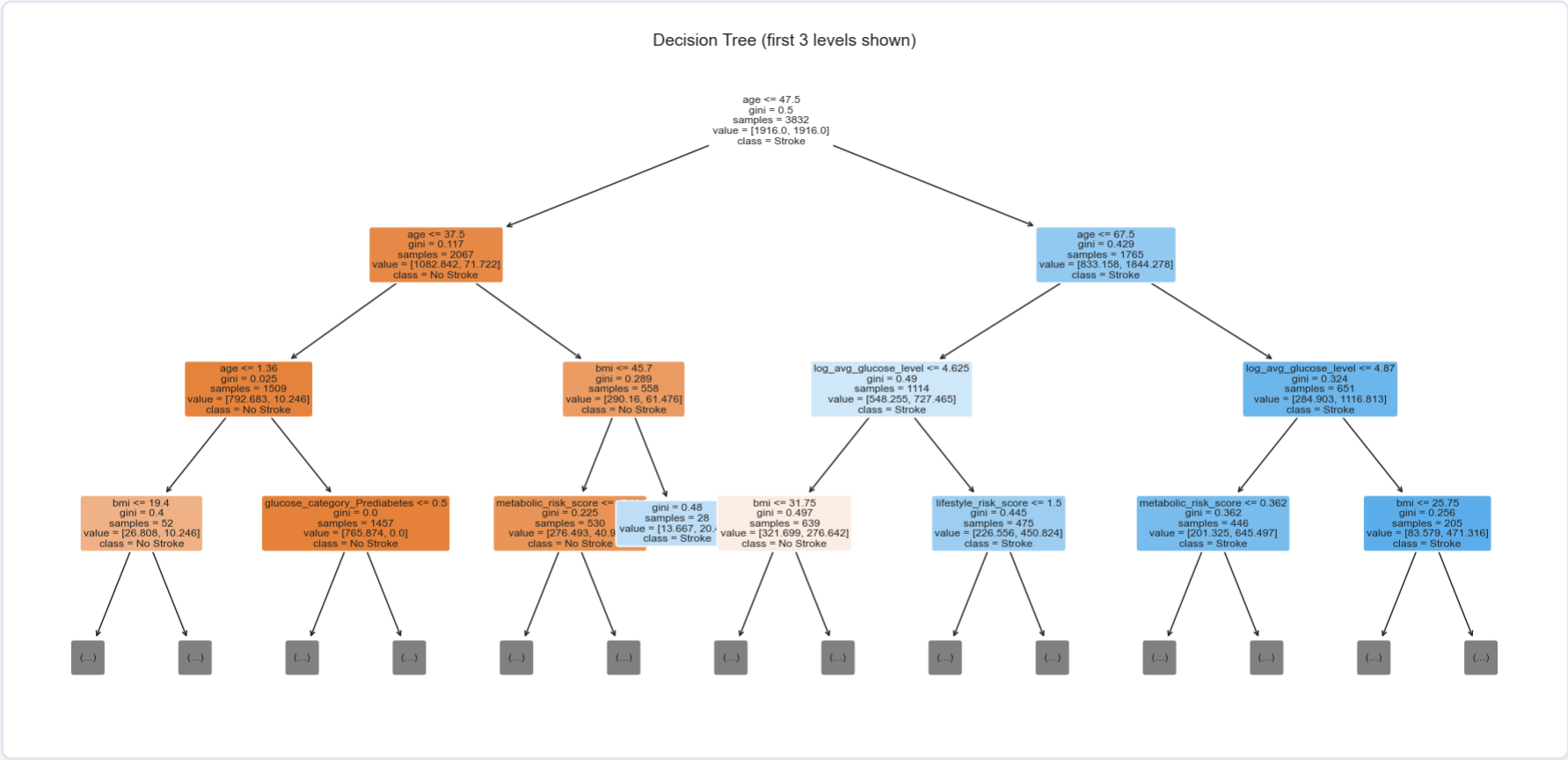
- Guarantees the same % of stroke vs. no-stroke patients in both train and test.
- Without it, you could get unlucky and have almost no positive cases in your test set.
- Critical for any imbalanced clinical outcome — always stratify by the target.

D.2 — Decision Tree: Key Hyperparameters

A Decision Tree learns a series of yes/no questions (e.g. "is age > 62?") that split the data into purer and purer groups.

Parameter	Controls	Tuning Intuition
max_depth	Max number of question-levels	Too small → underfits. Too large → overfits. Start 4–8.
min_samples_leaf	Min samples per leaf	Prevents tiny, unreliable leaves from outlier patients.
class_weight	Rebalances rare class	Set "balanced" — stroke is rare!
criterion	Split quality measure	gini (default, fast) vs. entropy (marginally more informative).

The Trained Decision Tree (first 3 levels)



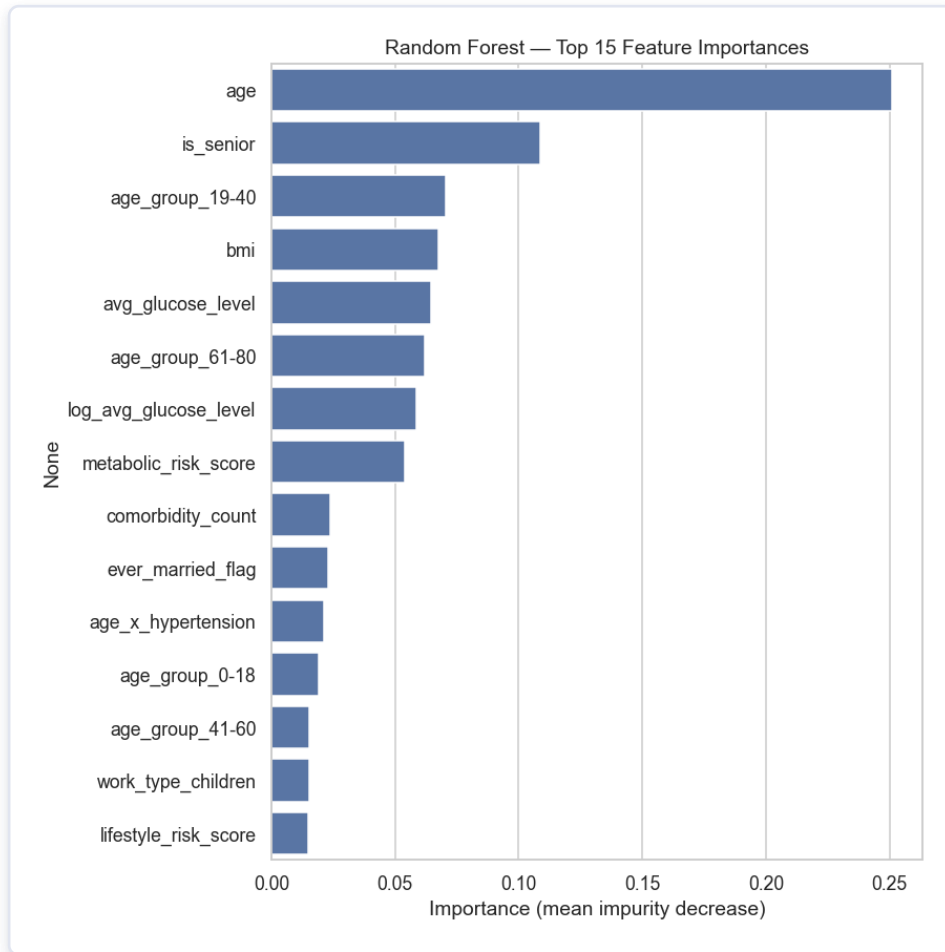
Each box asks a yes/no question about one feature; color intensity shows class purity. Notice how age and glucose dominate the earliest, most powerful splits.

D.3 — Random Forest: Key Hyperparameters

A Random Forest trains many decision trees on random subsets of rows/features, then averages their votes — usually far more stable than a single tree.

Parameter	Controls	Tuning Intuition
n_estimators	Number of trees	More trees = more stable, diminishing returns after a few hundred.
max_depth	Depth of each tree	Same intuition as the single Decision Tree.
max_features	Features considered per split	Lower = more diverse (less correlated) trees.
class_weight	Rebalances rare class	Keep "balanced" on for imbalanced clinical outcomes.

D.4 — Which Engineered Features Actually Matter?



The moment of truth for Part C

- age dominates — matching decades of stroke epidemiology literature.
- Engineered features like `metabolic_risk_score` and `comorbidity_count` often rank near the top — confirming domain-knowledge feature engineering added real signal.
- A low-ranking engineered feature is a normal, honest research finding too — not every idea works, and reporting that is good science.
- Cross-check importance against simple correlation for a robustness sanity check.

D.5 — The Metrics You Need for Any Clinical ML Report

TP = True Positive · TN = True Negative · FP = False Positive · FN = False Negative

Accuracy

$$(TP + TN) / (TP + TN + FP + FN)$$

Overall % correct — misleading alone on imbalanced data like ours (~95% no-stroke).

Precision

$$TP / (TP + FP)$$

Of all patients predicted "stroke," how many really had one?

Recall (Sensitivity)

$$TP / (TP + FN)$$

Of all real stroke patients, how many did we catch? Usually the most clinically important metric.

F1-score

$$2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$$

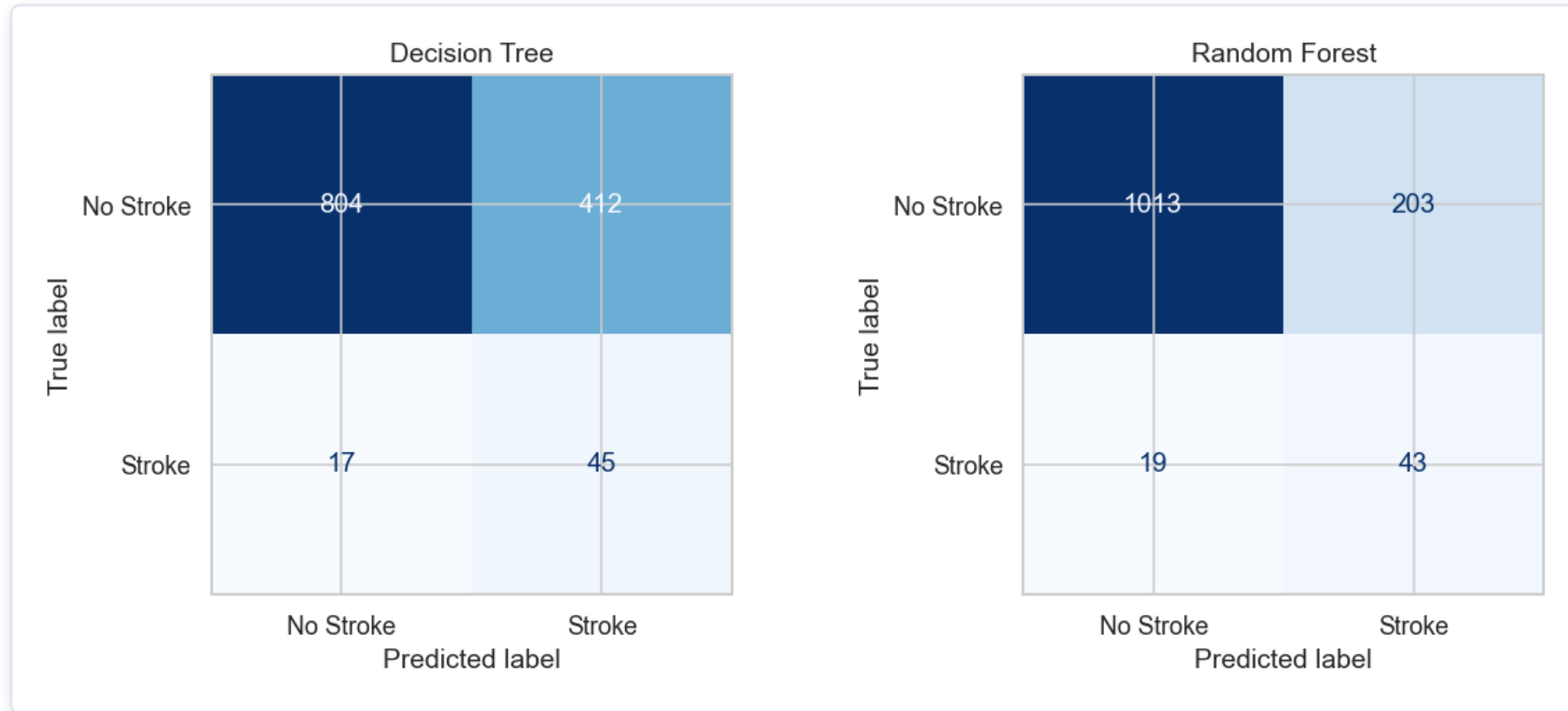
Harmonic mean of Precision & Recall — one balanced number.

ROC-AUC

$$\text{Area under the ROC curve (0.5 – 1.0)}$$

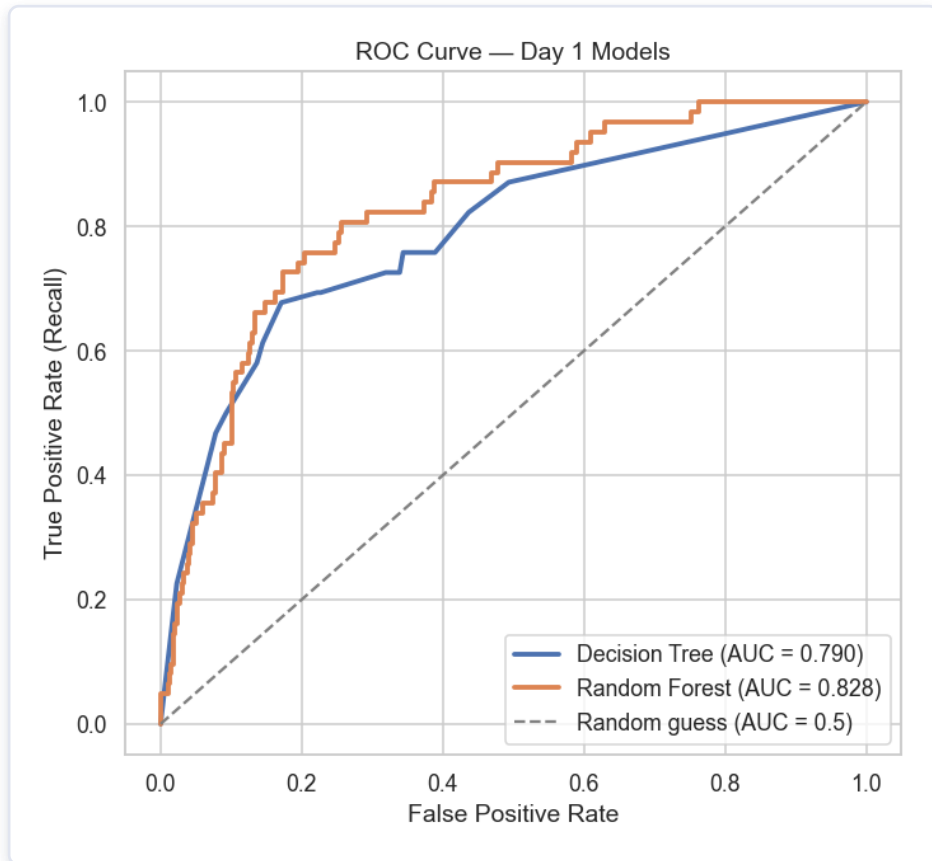
Ranking quality across every possible decision threshold, independent of any single cut-off.

Confusion Matrices — Decision Tree vs. Random Forest



Rows = actual class, columns = predicted class. In a screening context, a False Negative (missed real stroke risk) is usually far more dangerous than a False Positive.

ROC Curves — Day 1 Models



Reading the ROC curve

- The curve plots True Positive Rate vs. False Positive Rate across every threshold.
- The dashed diagonal is a random-guess baseline (AUC = 0.5).
- Higher and further from the diagonal = a better ranking model, regardless of the exact cut-off chosen.
- `class_weight="balanced"` plus these metrics give a far more honest picture than accuracy alone.

Day 1 Recap — What Is Supervised Learning?

- ✓ Supervised learning = learning a mapping $X \rightarrow y$ from a labeled training set, to predict y for new patients.
- ✓ Data ingestion → loaded and understood the KKU Stroke Dataset with pandas.
- ✓ EDA → distributions, missingness, imbalance, relationships with the target.
- ✓ Feature engineering → imputation, clinical binning, encoding, composite risk scores, skew correction.
- ✓ Stratified split → identical class balance across train/test.
- ✓ Model training → Decision Tree (interpretable) and Random Forest (ensemble, usually stronger).
- ✓ Feature importance → validated which engineered features actually carried signal.
- ✓ Evaluation → Accuracy, Precision, Recall, F1, ROC-AUC, ROC curve, confusion matrix.

See You Tomorrow for Day 2! 🙌

Tomorrow: Logistic Regression from first principles, the sigmoid neuron that bridges classical ML to Deep Learning, hands-on PyTorch neural networks (including a 1D CNN), and a closing guide to responsible AI research in public health.

Teerapong Panboonyuen, Ph.D. (Kao) · College of Computing, Khon Kaen University